

PS-4

1.a BCD

	A	B	C	D	a	b	c	d	e	f	g	DP
0	0	0	0	0	1	1	1	1	1	1	0	0
1	0	0	0	1	0	1	1	0	0	0	0	0
2	0	0	1	0	1	1	0	1	1	0	1	0
3	0	0	1	1	1	1	1	1	0	0	1	0
4	0	1	0	0	0	1	1	0	0	1	1	0
5	0	1	0	1	1	0	1	1	0	1	1	0
6	0	1	1	0	1	0	1	1	1	1	1	0
7	0	1	1	1	1	1	1	0	0	0	0	0
8	1	0	0	0	1	1	1	1	1	1	1	1
9	1	0	0	1	1	1	1	0	0	1	1	0

b.

For (a)

$$\text{MSOP: } A + C + BD + \bar{B}\bar{D}$$

$$\text{MPOS: } (\bar{B} + C + D) (A + B + C + D)$$

		AB			
		00	01	11	10
CD	00	1	0	d	1
	01	0	1	d	1
	11	1	1	d	d
	10	1	1	d	d

For (b)

MSOP: $\bar{B} + CD + \bar{C}\bar{D}$

MPOS: $(\bar{B} + C + D)(\bar{B} + \bar{C} + D)$

		AB			
		00	01	11	10
CD	00	1	1	d	1
	01	1	0	d	1
	11	1	1	d	d
	10	1	0	d	d

For (c)

MSOP: $B + \bar{C} + D$

MPOS: $B + \bar{C} + D$

		AB			
		00	01	11	10
CD	00	1	1	d	1
	01	1	1	d	1
	11	1	1	d	d
	10	0	1	d	d

For (d)

MSOP: $B\bar{C}D + \bar{B}D + \bar{B}C + C\bar{D}$

MPOS: $(B + C + \bar{D})(\bar{B} + C + D)(\bar{B} + \bar{C} + \bar{D})$

		AB			
		00	01	11	10
CD	00	1	0	d	1
	01	0	1	d	0
	11	1	0	d	d
	10	1	1	d	d

For (e)

MSOP: $\bar{B}\bar{D} + C\bar{D}$

MPOS: $(\bar{B} + C)\bar{D}$

		AB			
		00	01	11	10
CD	00	1	0	d	1
	01	0	0	d	0
	11	0	0	d	d
	10	1	1	d	d

For (f)

MSOP: $A + B\bar{C} + B\bar{D} + \bar{C}\bar{D}$

MPOS: $(A + B + \bar{D})(B + \bar{C})(\bar{C} + \bar{D})$

		AB			
		00	01	11	10
CD	00	1	1	d	1
	01	0	1	d	1
	11	0	0	d	d
	10	0	1	d	d

For (g)

MSOP: $A + B\bar{C} + \bar{B}C + B\bar{D}$

MPOS: $(A+B+C)(\bar{B} + \bar{C} + D)$

		AB			
		00	01	11	10
CD	00	0	1	d	1
	01	0	1	d	1
	11	1	0	d	d
	10	1	1	d	d

For (DP)

MSOP: $A\bar{D}$

MPOS: $(A)(\bar{D})$

		AB			
		00	01	11	10
CD	00	0	0	d	1
	01	0	0	d	0
	11	0	0	d	d
	10	0	0	d	d

C.

```

1 // Dataflow verilog
2
3 module SevenSegDataFlow(A,B,C,D,a,c,b,d,e,f,g,DP)
4 input A,B,C,D;
5 output a,b,c,d,e,f,g,DP;
6 assign a = A|C|(B&D)|(~B&~D);
7 assign b= ~B|(~C&~D)|(C&D);
8 assign c= B|~C|D;
9 assign d= (B&~C&D)|(~B&~D)|(~B&C)|(C&~D);
10 assign e= (~B&~D)|(C&~D);
11 assign f= A|(B&~C)|(B&~D)|(~C&~D);
12 assign g= A|(~B&C)|(B&~C)|(B&~D);
13 assign DP= (A&~D);
14 endmodule
15

```



d.

```
// Behavioral verilog

module SevenSegBehavioral(A,B,C,D,a,c,b,d,e,f,g,DP)
input A,B,C,D
output res[0:7]out;
always@(*)
begin
    case({A,B,C,D})
        4'b0000: out = 8'b11111100; // Digit 0
        4'b0001: out = 8'b01100000; // Digit 1
        4'b0010: out = 8'b11011010; // Digit 2
        4'b0011: out = 8'b11110010; // Digit 3
        4'b0100: out = 8'b01100110; // Digit 4
        4'b0101: out = 8'b10110110; // Digit 5
        4'b0110: out = 8'b10111110; // Digit 6
        4'b0111: out = 8'b11100000; // Digit 7
        4'b1000: out = 8'b11111111; // Digit 8
        4'b1001: out = 8'b11100110; // Digit 9
    endcase
end
```

e.

```
module Sevenseg Behavioral;
    reg A,B,C,D;
    wire [0:7]out;
```

Sevenseg Behavioral seg (A,B,C,D,out);

```
initial #100 $Finish
initial Begin
    A = 1'b0 ; B = 1'b0 ; C = 1'b0 ; D = 1'b0 ;
#10 A = 1'b0 ; B = 1'b0 ; C = 1'b0 ; D = 1'b1 ;
```

```

#10 A = 1'b0; B = 1'b0; C = 1'b1; D = 1'b0;
#10 A = 1'b0; B = 1'b0; C = 1'b1; D = 1'b1;
#10 A = 1'b0; B = 1'b1; C = 1'b0; D = 1'b0;
#10 A = 1'b0; B = 1'b1; C = 1'b0; D = 1'b1;
#10 A = 1'b0; B = 1'b1; C = 1'b1; D = 1'b0;
#10 A = 1'b0; B = 1'b1; C = 1'b1; D = 1'b1;
#10 A = 1'b1; B = 1'b0; C = 1'b0; D = 1'b0;
#10 A = 1'b1; B = 1'b0; C = 1'b0; D = 1'b1;
end

```

initial Begin

```

$ display ("A\TB\TC\TD\Ta\Tb\Tc\Td\Te\Tf\Tg\Tdp");
$ monitor (" %b\T%b\T%b\T%b\T%b\T%b\
            T%b\T%b\T%b\T%b\T%b\T%b",

```

```

A, B, C, D, out[0], out[1], out[2], out[3], out[4],
                        out[5], out[6], out[7]);

```

```

$ dumpfile ("Dump.vcd");

```

```

$ dump vars (0);

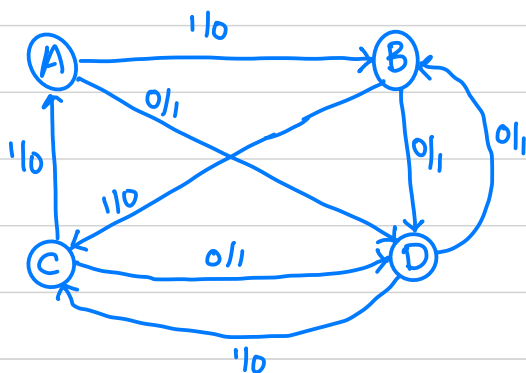
```

end

endmodule

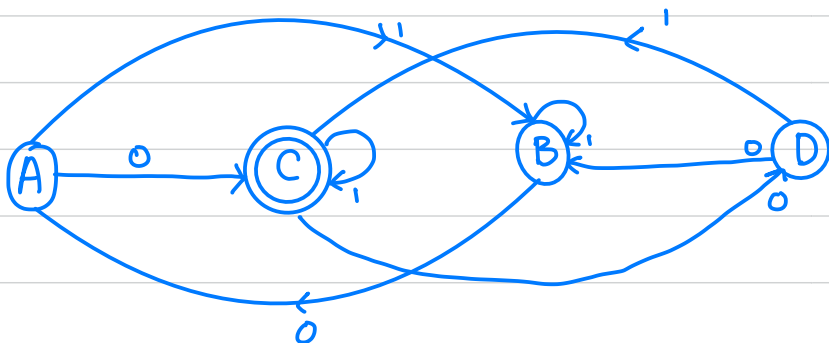
2.

State diagram is



It is a Mealy state machine

3.



This is a moore state machine

4. The initial state is A

String		0	1	1	0	1
State	A	D	C	A	D	C
Output		1	0	0	1	0

The output sequence will be 10010.

5. Verilog for the Moore machine from #3

```
module MooreMachine (BinInput, reset, clock, output);  
    input BinInput, reset, clock;  
    output reg out;  
    reg [1:0] nextstate, currstate;  
    Parameter A = z'd0, B = z'd1, C = z'd2, D = z'd3;
```

```
always @ (BinInput, reset, currstate)  
    Begin  
        if (reset)
```

nextstate = A;

else

Begin

case (currstate)

A: if (Bin Input == 0)

nextstate = C;

else

nextstate = B;

B: if (Bin Input == 0)

nextstate = A;

else

nextstate = B;

C: if (Bin Input == 0)

nextstate = D;

else

nextstate = C;

D: if (Bin Input == 0)

nextstate = B;

else

nextstate = C;

endcase

end

end

always @ (posedge clock)

begin

currstate C = nextstate;

case (nextstate)

A: out C = 0;

B: out C = 0;

C: out C = 1;

D: out C = 0;

endcase

end

end